Propiedades de Acceso (Getters y Setters)

Las propiedades vistas hasta ahora son propiedades de datos. Pero también **existen las propiedades de acceso** para asignar/retornar valores. Desde el exterior se ven igual que una propiedad normal. Dichos métodos **se usan habitualmente para validar el acceso** a las propiedades o para **crear propiedades virtuales**:

```
usuario = {
    nombre:"Iñigo",
    apellidos:"Chueca",
    get nombreCompleto(){
        return `${this.nombre} ${this.apellidos}`;
    }
}
>{nombre: "Iñigo", apellidos: "Chueca"}
usuario.nombreCompleto
>"Iñigo Chueca"
usuario.nombreCompleto = "Carlos Marín"; (Esto no funciona porque la propiedad nombreCompleto es un método de acceso, pero solo tiene get no set)
>"Carlos Marín"
usuario
>{nombre: "Iñigo", apellidos: "Chueca"}
```

Podemos añadir un set :

```
usuario = {
    nombre:"Iñigo",
    apellidos:"Chueca",
    get nombreCompleto(){
        return `${this.nombre} ${this.apellidos}`;
    },
    set nombreCompleto(nombreCompleto){
        [this.nombre,this.apellidos] = nombreCompleto.split(" ");
    }
}
>{nombre: "Iñigo", apellidos: "Chueca"}
usuario.nombreCompleto = "Carlos Marín";
>"Carlos Marín"
```

```
usuario
```

```
>{nombre: "Carlos", apellidos: "Marín"}
```

NOTA: Lógicamente **no** se puede aplicar el operador `delete` a las propiedades de acceso

Las propiedades de acceso también **se pueden emplear como decoradoras de propiedades de datos**. De este modo se puede controlar el acceso a las mismas. Lo que **no es posible es evitar que la propiedad sea visible**, pero se suele emplear como nombre `_nombre` para **indicar que dicha propiedad no debiera ser accedida directamente**: (Esto sería como las variables privadas en Java y sus método públicos getter, setter. Con esto protegemos el acceso a las variables aunque en la realidad si podríamos acceder aunque empiecen por _. Esto es un acuerdo que se ha llegado en la comunidad de JavaScript).

Por ejemplo:

```
let persona = {
  _edad: 5,
  nombre: "Juan",
  get edad() {
    return this._edad;
  },
  set edad(edad) {
    if (edad > 18) {
      this._edad = edad;
    }
  }
}
```

Herencia con constructores

NOTA: A pesar de que la herencia se puede usar con prototipos. Este tema de herencia se verá usando clases (clausula extends como en Java) en vez de prototipo en el siguiente tema(clases).

Para información más detallada acceder al enlace:

https://developer.mozilla.org/es/docs/Learn/JavaScript/Objects/Inheritance